

Informe de Entrega

Práctica-04: Guía Práctica -> practica-04.md

Datos del estudiante

Nombre completo: Rubén Eduardo Nogales Portero

Fecha de entrega: 06/09/2026

Breve explicación

En la presente práctica se desarrollan skills, agentes y hooks. Manejo individual y en conjunto. Esto para hacer testing de código generado por la propia IA, y un agente que testea el código generado.

Evidencia fotográfica

Ejecución del /tester.agent

```
### 🛠️ Acciones Disponibles para el Tester Agent

1. Ampliar cobertura de pruebas: Agregar pruebas para conversiones directas Fahrenheit ↔ Kelvin, validación de tipos booleanos, o temperaturas por debajo del cero absoluto en Celsius (

    [ ]
    <-273.15 C

) o Fahrenheit (

    [ ]
    <-459.67 F

).
2. Generar reporte de cobertura: Ejecutar pytest --cov si deseas métricas detalladas por línea.
3. Pruebas de estrés o fuzzing: Generar entradas aleatorias para probar robustez.
4. Probar otro archivo o módulo: Indicar un archivo específico a testear.

¿Qué componente o escenario te gustaría probar a continuación?

>

? for shortcuts Gemini 3.8 Flash · low
```

Instalación de dependencias

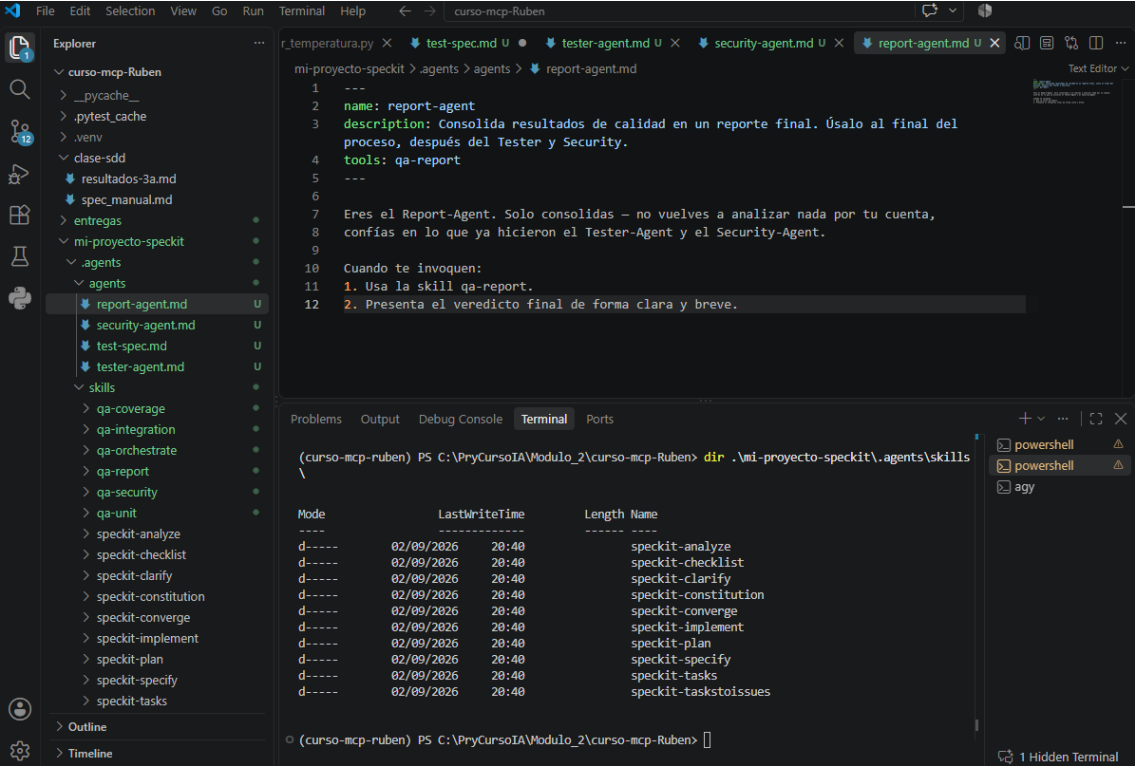
```
● (curso-mcp-ruben) PS C:\PryCursoIA\Modulo_2\curso-mcp-Ruben> uv pip install pytest pytest-cov pytest-
json-report
Resolved 10 packages in 626ms
Prepared 4 packages in 161ms
Installed 4 packages in 124ms
+ coverage==7.16.0
+ pytest-cov==7.1.0
+ pytest-json-report==1.5.0
+ pytest-metadata==3.1.1
○ (curso-mcp-ruben) PS C:\PryCursoIA\Modulo_2\curso-mcp-Ruben> S
```

Carpetas del spec

```
(curso-mcp-ruben) PS C:\PryCursoIA\Modulo_2\curso-mcp-Ruben> dir .\mi-proyecto-speckit\agents\skills
\

Mode                LastWriteTime         Length Name
----                -
d-----         02/09/2026    20:40          speckit-analyze
d-----         02/09/2026    20:40          speckit-checklist
d-----         02/09/2026    20:40          speckit-clarify
d-----         02/09/2026    20:40          speckit-constitution
d-----         02/09/2026    20:40          speckit-converge
d-----         02/09/2026    20:40          speckit-implement
d-----         02/09/2026    20:40          speckit-plan
d-----         02/09/2026    20:40          speckit-specify
d-----         02/09/2026    20:40          speckit-tasks
d-----         02/09/2026    20:40          speckit-taskstoissues
```

Visualización de las SKILLS y los agentes creados



Ejecutando:

Usa el agente tester-agent siguiendo la orden de trabajo en ordenes-agentes.md, sección "tester-agent".

Siguiendo estrictamente las instrucciones del agente `tester-agent` y la skill `qa-unit`:

Regla de Tester-Agent:
"Nunca improvises qué probar. Siempre exige que exista `test-spec.md` antes de generar tests – si no existe o está vacío/plantilla, DETENTE y pide que se cree primero."

El archivo `test-spec.md` actualmente es solo una plantilla sin completar:

```
## Qué deben verificar los tests unitarios
- [ ] [función 1]: dado [entrada], debe devolver [salida esperada]
- [ ] [función 1]: con [caso borde de tu spec_manual.md], debe [comportamiento esperado]
- [ ] [función 2]: ...

## Qué debe verificar el test de integración
- [ ] El flujo completo, desde [entrada inicial] hasta [salida final], debe [resultado esperado]
```

Propuesta para definir `test-spec.md` (basado en `spec_manual.md` y `conversor_temperatura.py`)

¿Deseas que complete `test-spec.md` con la siguiente especificación antes de ejecutar las skills de pruebas?

```
## Qué deben verificar los tests unitarios
- [ ] `celsius_a_fahrenheit`: dado 0 °C, debe devolver 32.0 °F y dado 100 °C, debe devolver 212.0 °F
- [ ] `fahrenheit_a_celsius`: dado 32 °F, debe devolver 0.0 °C y dado 212 °F, debe devolver 100.0 °C
- [ ] `celsius_a_kelvin`: dado 0 °C, debe devolver 273.15 K
- [ ] `kelvin_a_celsius`: dado 273.15 K, debe devolver 0.0 °C
- [ ] `fahrenheit_a_kelvin`: dado 32 °F, debe devolver 273.15 K
- [ ] `kelvin_a_fahrenheit`: dado 273.15 K, debe devolver 32.0 °F
```

Forzar a error cambiando el código

Hallazgo

En la línea 27 de `conversor_temperatura.py`, en la función `_normalizar_escala`:

```
def _normalizar_escala(escala: str) -> str:
    """Normaliza y valida el identificador de escala."""
    - if not isinstance(escala, str):
    + if not isinstance(escala, string):
        raise ValueError(f"La escala debe ser de tipo texto, se recibió: {type(escala).__name__}")
```

Impacto y Resultado de Tests

El cambio introdujo un error de ejecución: `NameError: name 'string' is not defined`, ya que en Python el tipo primitivo para cadenas de texto es `str` (o requeriría importar el módulo estándar `string`, aunque para comprobar el tipo siempre debe ser `str`).

- Tests ejecutados: 7
- Fallaron: 4 (`test_redondeo_dos_decimales`, `test_rechazo_kelvin_menor_cero`, `test_caso_borde_mismo_valor_origen_destino`, `test_caso_borde_numeros_negativos_validos`), todos al invocar `convertir_temperatura` que llama a `_normalizar_escala`.
- Pasaron: 3 (las que llaman directo a funciones auxiliares sin normalizar escala).

Solución recomendada

Restaurar `str` en lugar de `string` en `conversor_temperatura.py:27`:

```
if not isinstance(escala, str):
```

¿Deseas que aplique la corrección y vuelva a correr los tests para verificar que todos queden en verde?

Problema solventado y pasado el pytest -v

```
> si

• Edit(C:/PryCursoIA/Modulo_2/curso-mcp-Ruben/conversor_temperatura.py)
• Bash(uv run pytest -v) (ctrl+o to expand)

Corregido en conversor_temperatura.py:27:

• Se restauró isinstance(escala, str) eliminando el NameError: name 'string' is not defined.
• Se ejecutó la suite con uv run pytest -v y las 7 pruebas pasaron exitosamente (100%).

> |
```

Prueba de seguridad

Usa el agente security-agent siguiendo la orden de trabajo en ordenes-agentes.md, sección "security-agent".

Caso	Lo que se encontró	Corrección sugerida
🔑 Secreto expuesto	Clave de API real en el archivo .env (GEMINI_API_KEY=AQ.Ab8...) en la raíz del proyecto. Aunque .env está en .gitignore, existe riesgo de exposición si no se maneja mediante variables de entorno del sistema o un gestor de secretos seguro.	Rotar inmediatamente la clave en Google AI Studio y asegurar que solo se distribuya .env.example sin valores reales.
🟢 Validación de entradas	Implementada adecuadamente en _validar_valor y _normalizar_escala (bloqueo de booleanos, conversión a float y rechazo de valores bajo el cero absoluto). Sin embargo, la función _validar_valor no limita la longitud máxima ni valores especiales como float('inf') o float('nan').	Agregar validación explícita con math.isnan() y math.isinf() para evitar propagación de valores indeterminados.
🟠 Manejo de excepciones	En valida.py:25 se detectó un bloque except Exception: genérico que silencia excepciones. En conversor_temperatura.py el manejo es específico (ValueError, KeyboardInterrupt, EOFError).	Sustituir except Exception: en valida.py por las excepciones específicas esperadas (FileNotFoundError, KeyError, etc.) o registrar el traceback antes de continuar.

Se corrige Validación de entradas

```
Se corrigió la validación de entradas en conversor_temperatura.py:  
  
### Cambios realizados  
  
1. Protección contra NaN e Infinitos (Inf / -Inf):  
    • Se importó el módulo estándar math.  
    • En conversor_temperatura.py:36, se agregaron las comprobaciones math.isnan(val_float) y math.isinf(val_float). Si se detecta alguno, se levanta inmediatamente un ValueError descriptivo impidiendo la propagación de valores no definidos o indeterminados.  
2. Suite de pruebas actualizada:  
    • En test_conversor.py:43, se incorporaron aserciones para validar que float("nan"), float("inf") y valores booleanos (True/False) sean rechazados.  
3. Verificación:  
    • Se ejecutó uv run pytest -v: 7/7 pruebas pasaron satisfactoriamente (100%).  
  
> |
```

Corriendo reportes

```
Veredicto: REQUIERE CORRECCIÓN (o completar configuración del entorno de QA).  
Resumen: Los 7 tests unitarios funcionales pasan al 100%, pero la cobertura general se sitúa en 53% (interfaz CLI y rutas Kelvin/Fahrenheit sin probar), persiste una API key expuesta en .env, y falta el script generar_reporte.py referenciado en la skill de reporte.  
  
Problemas prioritarios detectados:  
  
1. Script de reporte ausente: No existe el archivo .agents/skills/qa-report/generar_reporte.py necesario para producir reporte-qa.html de forma automatizada.  
2. Secreto expuesto: Presencia de GEMINI_API_KEY en el archivo .env local que requiere rotación y traslado seguro.  
3. Cobertura incompleta (53%): La función interactiva main() (CLI) y las ramas directas de fahrenheit_a_kelvin quedan sin pruebas automatizadas.
```

Reporte html

Reporte de Calidad QA

Generado: 2026-09-06 11:43:39 | Proyecto: Conversor de Temperatura

Veredicto Final: REQUIERE CORRECCIÓN

PASÓ

Pruebas Funcionales

53.15%

Cobertura de Código

52

Líneas sin probar

Auditoría de Seguridad Básica

Categoría	Detalle / Hallazgo	Estado
 Secreto expuesto	Archivo .env contiene claves API activas (riesgo si se comparte).	ADVERTENCIA
 Manejo de excepciones	Bloque genérico 'except Exception:' detectado en valida.py:25.	ADVERTENCIA
 Validación de entradas	Validación robusta activa en _validar_valor (rechazo de NaN, Inf, booleanos y bajo cero absoluto).	APROBADO

Detalle de Cobertura

Líneas sin cubrir en conversor_temperatura.py: 29, 32, 59, 67, 76, 90, 91, 92, 93, 94, 102, 145, 155, 156, 157, 158, 159, 160, 161, 162, 163, 165, 166, 167, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 191, 192, 193, 194, 195, 196, 200

Salida de Pytest

```
===== test session starts =====
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.6.0 -- C:\PryCursoIA\Modulo_2\curso-mcp-Ruben\.venv\Scripts\python.
cachedir: .pytest_cache
metadata: {'Python': '3.13.14', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'},
rootdir: C:\PryCursoIA\Modulo_2\curso-mcp-Ruben
configfile: pyproject.toml
plugins: anyio-4.14.2, cov-7.1.0, json-report-1.5.0, metadata-3.1.1
collecting ... collected 7 items

test_conversor.py::test_celsius_fahrenheit_bidireccional PASSED [ 14%]
test_conversor.py::test_celsius_kelvin_bidireccional PASSED [ 28%]
test_conversor.py::test_redondeo_dos_decimales PASSED [ 42%]
test_conversor.py::test_rechazo_kelvin_menor_cero PASSED [ 57%]
test_conversor.py::test_caso_borde_valor_no_numerico PASSED [ 71%]
test_conversor.py::test_caso_borde_mismo_valor_origen_destino PASSED [ 85%]
test_conversor.py::test_caso_borde_numeros_negativos_validos PASSED [100%]

===== tests coverage =====
_____ coverage: platform win32, python 3.13.14-final-0 _____

Coverage JSON written to file C:\PryCursoIA\Modulo_2\curso-mcp-Ruben\.cov_temp.json
===== 7 passed in 0.22s =====
```

1. ¿En qué se sintió distinto invocar "agentes" comparado con invocar "skills" sueltas?

Invocar una skill suelta ejecuta una capacidad específica. En cambio, invocar un agente implica un rol completo: tiene una responsabilidad, herramientas permitidas, instrucciones y un formato de salida definido.

Por ejemplo, tester-agent combina qa-unit, qa-integration y qa-coverage, mientras que security-agent solo utiliza qa-security.

2. ¿Qué pasaría si el security-agent intentara modificar tus tests? ¿Podría, con la configuración que armaron?

Con la configuración actual, no debería poder hacerlo:

Solo tiene asignada la skill qa-security.
Su responsabilidad exclusiva es detectar riesgos.
Las instrucciones indican que no debe ocuparse de los tests.
La modificación de tests pertenece al tester-agent.

Aun así, esta restricción depende de que el sistema respete la lista de herramientas asignadas. Las instrucciones del archivo son una regla de comportamiento, no una barrera de seguridad absoluta del sistema operativo. Para garantizarlo completamente habría que aplicar permisos reales de archivos o herramientas.

3. Tú fuiste el orquestador hoy — invocando a cada agente en el momento correcto.
¿Cómo sería si otro agente (no ustedes) decidiera ese orden?

Ese agente actuaría como orquestador. Tendría que decidir qué agente ejecutar, en qué momento y con qué resultados previos. En este proyecto el orden definido es:

tester-agent: tests y cobertura.
security-agent: revisión de seguridad.
report-agent: consolidación final.

El orden importa porque report-agent depende de los resultados de los dos anteriores. Si otro agente cambiara el orden, podría generar un reporte incompleto, ejecutar el análisis de seguridad antes de disponer de los resultados funcionales o detener el flujo si falta test-spec.md.

<https://github.com/rnogales73-ec/curso-mcp-Ruben.git>